SOFTWARE REQUIREMENTS

SPECIFICATION (SRS)

“AlumniConnect - Student & Alumni Mentoring Portal”

for Student–Alumni Mentorship

Project Name: AlumniConnect
A full-stack web portal connecting students with alumni mentors

1. Introduction

The Software Requirements Specification (SRS) document describes the detailed requirements for AlumniConnect, a web-based student and alumni mentoring portal. The system enables students to register, maintain their academic profile, search for suitable alumni mentors, and submit one-on-one mentorship requests. Alumni can register as mentors and provide professional information such as company, designation, industry, skills, experience, biography, and mentee capacity.

1.1 Objective of the Programme

The objective of this document is to clearly define the functional and non-functional requirements of the AlumniConnect system. It serves as a guide for developers, testers, and stakeholders involved in the project and provides a common understanding of the system's expected behaviour.

1.2 Scope

The application is a web-based mentoring platform that provides:

- A registration and profile management interface for students and alumni mentors.
- Secure authentication with role-based access for STUDENT and MENTOR users.
- A searchable directory of alumni mentors with department, industry, and experience filters.
- A mentorship request workflow in which students select a goal and write an introductory note.
- A centralized relational database for user, student, and alumni profile information.
- Input validation, password-strength evaluation, and structured error reporting.

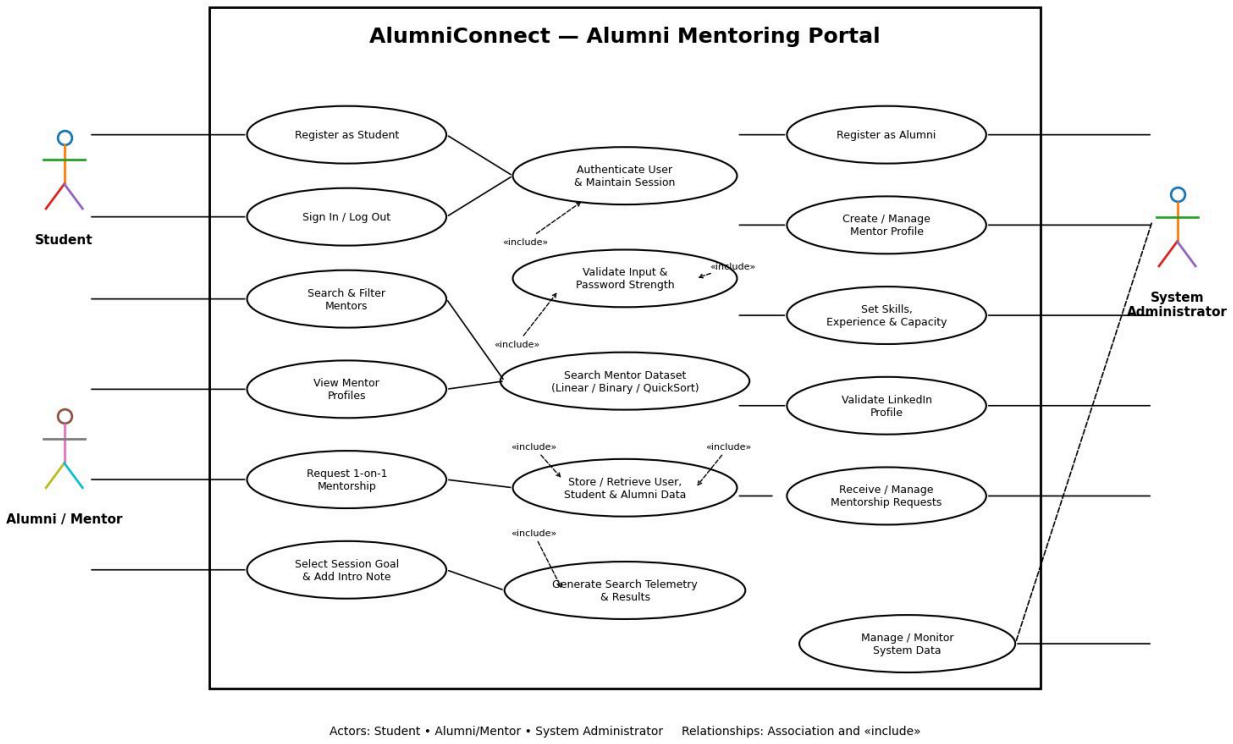
1.3 System Description

AlumniConnect facilitates interaction between students and verified alumni mentors through a single web portal. Students can create profiles and discover mentors based on professional and academic attributes. Alumni can create mentor profiles describing their professional background and mentoring capacity. The frontend communicates with a Java backend through HTTP-based APIs, while the backend accesses a MySQL/MariaDB database using JDBC.

1.4 Use Case Diagram

The Use Case Diagram represents the interactions between the main system actors—Student, Alumni/Mentor, and Unauthenticated Visitor—and the AlumniConnect portal. It covers registration, authentication, profile management, mentor search and filtering, viewing mentor details, and submitting mentorship requests.

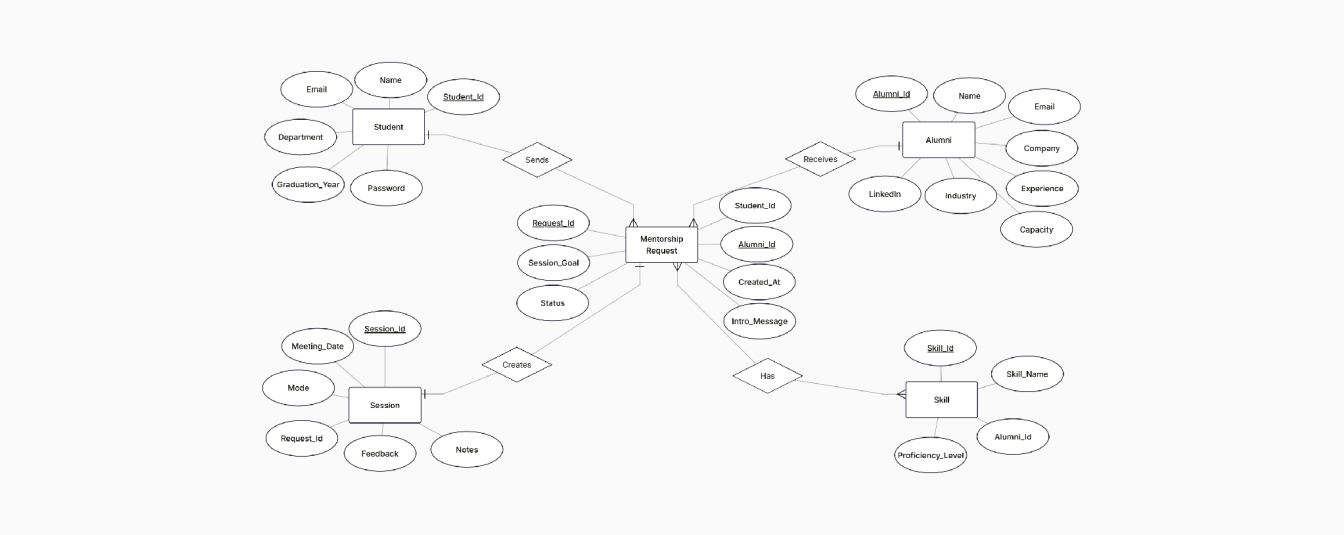
Use Case Diagram:



1.5 ER Diagram

The Entity-Relationship (ER) Diagram represents the relational data structure of AlumniConnect, including users and their associated student and alumni/mentor profile information, along with the relationships maintained through primary and foreign keys.

ER Diagram:



2. Input

The system accepts the following inputs from its users:

2.1 Student Inputs

Input Field	Description
Full Name	Student's full name.
Email Address	Unique email used for authentication and communication.
Mobile Number	Student contact number.
Password	Account password satisfying the defined strength rules.
Student ID	Academic identifier such as STU-2024-001.
Department	Student's academic department.
Graduation Year	Expected/current graduation year.
Mentor Search Query	Keywords for searching mentor profiles.
Department / Industry / Experience Filters	Optional criteria used to narrow mentor results.
Mentorship Goal	Career Guidance, System Design Prep, Resume Review, or Higher Studies.
Introductory Note	Personalized message accompanying a mentorship request.

2.2 Alumni Mentor Inputs

Input Field	Description
Full Name	Alumni mentor's full name.
Email Address	Unique email used for authentication.
Mobile Number	Alumni contact number.
Password	Account password satisfying the defined strength rules.
Department	Department from which the alumni graduated.
Graduation Year	Year of graduation.
Company	Current or associated organization.
Designation	Professional job title.
Industry	Industry sector in which the mentor works.
Years of Experience	Professional experience of the mentor.
Skills	Comma-delimited list of professional skills.
Bio	Short professional biography.
LinkedIn Profile	LinkedIn profile URL.
Maximum Mentees	Maximum number of mentees the mentor can support.

2.3 Authentication Inputs

Input	Description
Email	Registered account email.
Password	Account password.
Role	Student or Alumni/Mentor role represented by the authenticated profile.

3. Output

The system produces the following outputs:

3.1 For Students

- Successful registration and automatic onboarding to the mentor directory.
- A searchable list of available alumni mentors with relevant profile information.
- Filtered mentor results based on department, industry, and minimum experience.
- Individual mentor profile details.
- Mentorship request form with selectable session goals and an introductory note.
- Validation feedback for invalid registration, login, password, or profile data.
- Session state showing the student's name, avatar initial, and STUDENT role.

3.2 For Alumni Mentors

- Successful alumni/mentor registration and profile creation.
- Mentor profile containing professional and academic information.
- Session state showing the mentor's name, avatar initial, and MENTOR role.
- Validation feedback for invalid registration information.

3.3 System Responses & Reports

- Structured JSON responses for API requests.
- Password-strength response containing a score from 0–4, criterion flags, and missing suggestions.
- Structured HTTP 400 validation responses containing a success flag, message, and field-level errors.
- Mentor search telemetry showing dataset pool, matched count, comparison operations, and execution latency.
- Health/status response through the system health endpoint.

4. Algorithms Used

Algorithm / Process	Where Used	Description
Input Validation using Regular Expressions	Registration & authentication inputs	Checks names, emails, mobile numbers, student IDs, LinkedIn URLs and other field constraints before database operations.
Password Strength Evaluation	Password validation	Evaluates length, uppercase, lowercase, number and special-character criteria and produces a 0–4 score.
Manual Multi-Field Linear Search - O(N)	Mentor directory	Scans mentor attributes including name, company, designation, skills, department and bio; supports tokenized multi-word queries.
Quicksort - O(N log N) average	Mentor search	Sorts in-memory mentor data using selected keys such as name, company, graduation year or experience.
Binary Search - O(log N)	Mentor search	Performs divide-and-conquer lookup on sorted data, including exact/prefix matching and boundary expansion for duplicate keys.
Debounced Search	Frontend mentor search	Waits approximately 280 ms after typing stops before triggering filtering/search, reducing unnecessary requests.
Database Transactions	Registration	Supports consistent user/student registration and database integrity through JDBC transactions.
Session Persistence	Authentication	Stores the authenticated session/user profile in browser localStorage under alumniConnectUser.

5. Technologies Used

5.1 Frontend

Technology	Type	Purpose
HTML5	Markup Language	Structure of the web application and forms.
CSS3	Styling	Responsive layout, visual design, validation states, animations and UI components.
Vanilla ES6 JavaScript	Client-side scripting	Form handling, session state, API communication, live validation and mentor search.
Native Browser APIs	Web APIs	HTTP Fetch, localStorage and other browser-side functionality.

5.2 Backend

Technology	Type	Purpose
Java 17+ / OpenJDK	Programming Language / Runtime	Core server-side application logic.
Apache Maven 3.8+	Build Tool	Dependency management, compilation and test execution.
Java HttpServer	HTTP Server	Hosts REST-style HTTP endpoints and routes frontend requests.
org.json	JSON Library	Parsing and producing JSON request/response data.
JDBC	Database Connectivity	Connects the Java backend to MySQL/MariaDB and performs database operations.
TCP Socket Server	Network Service	Provides newline-delimited JSON registration communication on port 5002.
JUnit 5	Testing Framework	Automated testing of validation, search algorithms, JDBC and endpoints.

5.3 Database

Technology	Type	Purpose
MySQL / MariaDB	Relational DBMS	Centralized storage of users, student profiles and alumni profiles.
SQL	Query Language	Data insertion, retrieval and update operations.
InnoDB / Foreign Keys	Database Integrity	Maintains relational consistency between user and profile records.

6. Other Specifications

6.1 Users of the System

User Role	Who They Are	What They Can Do
Student	Undergraduate or graduate student using the mentoring portal	Register, sign in, maintain student information, search/filter mentors, view mentor details and submit mentorship requests.
Alumni / Mentor	Graduate/alumni professional offering mentorship	Register as a mentor, create a professional profile and participate in the mentoring platform.
Unauthenticated Visitor	Person accessing the portal without an active session	Access the authentication/registration interface; protected mentor-directory actions require authentication.

6.2 Non-Functional Requirements

- Performance: Mentor search and frontend interactions should remain responsive for normal project-scale datasets.
- Usability: The interface should be clean, simple, responsive and provide immediate validation feedback.
- Security: Authentication and protected application views must prevent unauthenticated access; passwords must not be stored as plain text.
- Reliability: Database operations should maintain data consistency using JDBC transactions.
- Maintainability: Frontend, server, database-access, model, validation and search components should remain separated into logical modules.
- Compatibility: The web interface should work with modern browsers such as Chrome, Firefox, Safari and Edge.
- Scalability: The API-based architecture should allow the backend to support additional clients or future application extensions.
- Testability: Core validation, search and JDBC functionality should be covered by automated tests.

6.3 Constraints

- The project uses a Java backend and JDBC-based relational database access.
- The frontend intentionally uses Vanilla HTML5, CSS3 and ES6 JavaScript without React or JSX.
- The system depends on a running MySQL/MariaDB database for persistent data.
- The current implementation is designed around a local/project deployment environment.
- Mentorship matching is based on searchable profile attributes and filters; an AI-based recommendation engine is not specified in the supplied project description.
- The available README specifies a mentorship request interface but does not specify a separate mentor approval/scheduling workflow.

7. System Features

7.1 Student Features

- Student can register using personal and academic information.
- Student ID is checked against the defined alphanumeric/hyphen/slash format.
- New student registration automatically establishes the user session and moves the user toward the mentor directory.
- Student can sign in and maintain an authenticated session.
- Student can search mentors by name, company, designation, skills, department and bio.
- Student can use department, industry and minimum-experience filters.
- Student can view individual mentor details.
- Student can initiate a one-on-one mentorship request by selecting a session goal and writing an introductory note.

7.2 Alumni Mentor Features

- Alumni can register with academic and professional information.
- Alumni can provide company, designation, department, graduation year, experience, industry, skills, bio and mentee capacity.
- LinkedIn profile URL syntax is validated during registration.
- Alumni can sign in and receive a MENTOR role indicator in the interface.

7.3 Authentication & Validation Features

- Unauthenticated users are directed to the Sign In view before protected mentor-directory or request actions.
- Session information is persisted in localStorage using the alumniConnectUser key.
- The header dynamically displays the user's avatar initial, full name, role badge and logout action.
- Full-name validation checks length, permitted characters and consecutive whitespace.
- Password validation checks minimum length and required character classes.
- A live four-level password-strength meter and requirement checklist provide immediate feedback.
- Password matching is checked in real time.
- The backend exposes a password-strength API endpoint.

7.4 Mentor Search Features

- Search is debounced by approximately 280 ms.
- Search supports multiple mentor attributes rather than only a single field.
- Manual linear search is available for broad multi-field matching.
- Quicksort and binary search are implemented for sorted-key searching.
- Search responses expose comparison counts and execution timing telemetry.

7.5 General System Features

- REST-style HTTP API communication between frontend and backend.
- Centralized relational data storage through MySQL/MariaDB.
- CORS headers and JSON request parsing are handled by backend utilities.
- A health-check endpoint is provided for system status.

8. Remark: Technology & Database Choice

8.1 Why Vanilla HTML/CSS/JavaScript?

Aspect	Explanation
No Build Dependency	The frontend can run without React, JSX or a complex build pipeline.
Direct Browser Support	HTML5, CSS3 and ES6 JavaScript are directly supported by modern browsers.
Learning Value	The implementation demonstrates core web concepts, DOM interaction, client-side validation and API communication.
Lightweight UI	The absence of a frontend framework keeps the client-side application small and straightforward.

8.2 Why a Java Backend with HTTP APIs?

Aspect	Explanation
Separation of Concerns	Frontend presentation and backend business/database logic are maintained separately.
Java Requirement	The backend demonstrates server-side programming, object-oriented design, JDBC and networking concepts.
Reusability	HTTP/JSON endpoints can serve the current web frontend and can support future clients.
Networking	The project also demonstrates a TCP socket server for newline-delimited JSON communication.

8.3 Why MySQL / MariaDB?

Aspect	Explanation
Structured Data	Users, students and alumni profiles have clearly defined relational attributes.
Data Integrity	Primary/foreign-key relationships help maintain consistency between user and profile records.
JDBC Integration	Java provides mature JDBC support for relational database connectivity.
Query Support	SQL provides direct retrieval and filtering of mentor records.

In summary, the selected stack — Vanilla HTML5/CSS3/ES6 JavaScript, Java with Maven/JDBC, and MySQL/MariaDB — provides a lightweight full-stack implementation while demonstrating web development, database management, validation, algorithms, networking and automated testing.

9. Testing & Conclusion

9.1 Automated Testing

The project includes a JUnit 5 automated testing suite covering core backend functionality. The supplied project information reports 48 tests with zero failures, zero errors and zero skipped tests.

Test Suite	Tests	Coverage
ValidationTest	25	Email, mobile, full name, password strength, password mismatch, student ID, graduation years, HTTP registration, login flow and password-strength endpoint.
SearchAlgorithmsTest	19	Linear search, case-insensitivity, Quicksort ordering, exact/prefix Binary Search, boundary expansion and timing metrics.
JDBCTest	4	JDBC driver loading, database connection stability and transactional integrity.

9.2 Conclusion

AlumniConnect is a web-based student–alumni mentoring portal intended to simplify the process of discovering and approaching alumni mentors. The system combines authenticated student and mentor profiles, structured relational data, strict input validation, multi-field mentor search and a mentorship-request interface. Its Java/JDBC backend, Vanilla HTML/CSS/JavaScript frontend, custom search algorithms and automated JUnit testing provide a practical full-stack implementation suitable for a college project.

This SRS defines the functional, non-functional, input, output, algorithmic, technological and testing requirements represented in the supplied AlumniConnect project description. Features not specified in the project description, such as automated mentor matching, chat, calendar scheduling, notifications, or mentor approval workflows, are outside the currently documented scope.